APRIL 2026

PROJECT REPORT

On

AI-Powered Stock Market Analysis Platform

Submitted by

ARYA SHAH (IU2241230098)

In fulfillment for the award of the degree

Of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA
WEB: www.indusuni.ac.in

PROJECT REPORT

ON

AI-Powered Stock Market Analysis Platform

AT

In the partial fulfillment of the requirement
for the degree of
Bachelor of Technology
in
Computer Science And Engineering

PREPARED BY

Arya Shah (IU2241230098)

UNDER GUIDANCE OF

External Guide
<Guide name>
(<Company Name>)

Internal Guide
Prof. Ruchi Patel
Assistant Professor,
Dept. of CSE, I.T.E,
Indus University, Ahmedabad

SUBMITTED TO

INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA
WEB: www.indusuni.ac.in

APRIL 2026

CANDIDATE'S DECLARATION

I declare that the final semester report entitled **"AI-Powered Stock Market Analysis Platform"** is my own work conducted under the supervision of the guide **Prof. Ruchi Patel**.

I further declare that to the best of my knowledge, the report for B.Tech final semester does not contain part of the work which has been submitted for the award of B.Tech Degree either in this university or any other university without proper citation.

Candidate's Signature
Arya Shah (IU2241230098)

Guide : Prof. Ruchi Patel
Assistant Professor
Department of Computer Science And Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY - Ahmedabad,
State: Gujarat

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING

COMPUTER SCIENCE AND ENGINEERING

2025 - 2026

CERTIFICATE

Date: 13-03-2026

This is to certify that the project work entitled **"AI-Powered Stock Market Analysis Platform"** has been carried out by **Arya Shah** under my guidance in partial fulfillment of degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING (Final Year)** of Indus University, Ahmedabad during the academic year 2025-2026.

Prof. Ruchi Patel Assistant Professor, Dept. of CSE, IITE, Indus University, Ahmedabad	Dr. Kaushal Jani Head of Department, Dept. of CSE, IITE, Indus University, Ahmedabad	Prof. Zalak Vyas Head of Department, Dept. of CSE, IITE, Indus University, Ahmedabad

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Prof. Ruchi Patel, Assistant Professor, Department of Computer Science And Engineering, Indus Institute of Technology and Engineering, Indus University, Ahmedabad, for her invaluable guidance, constant encouragement, and constructive criticism throughout the course of this project work.

I am deeply thankful to Dr. Kaushal Jani and Prof. Zalak Vyas, Head of Department, Computer Science and Engineering, for providing the necessary infrastructure and support to carry out this project successfully.

I extend my heartfelt thanks to the entire faculty of the Department of Computer Science And Engineering for their support and encouragement during the academic year 2025-2026.

I am also grateful to the open-source community for providing the frameworks and libraries (React, TensorFlow, FastAPI, Express.js, MongoDB) that made this project possible.

Finally, I express my deep gratitude to my family and friends for their unwavering support, patience, and motivation throughout this endeavor.

Arya Shah

IU2241230098

ABSTRACT

The rapid growth of retail participation in global stock markets has created a critical demand for intelligent, accessible, and real-time financial analysis tools. Traditional stock market platforms either provide raw data without actionable insights or charge prohibitive subscription fees for AI-driven analytics, creating a significant accessibility gap for individual investors. This thesis presents the design, development, and deployment of an AI-Powered Stock Market Analysis Platform - a full-stack web application that democratizes financial intelligence through deep learning.

Our approach integrates three independent microservices into a cohesive analytical ecosystem. First, a React 18 frontend delivers a responsive, dark-themed user interface featuring real-time market dashboards, interactive price charts built with Recharts, and a simulated paper trading engine that allows users to practice buy/sell strategies without financial risk.

Second, a Node.js/Express backend serves as the API gateway and business logic layer. It implements JWT-based authentication with bcrypt password hashing (12 salt rounds), manages user portfolios and watchlists through MongoDB via Mongoose ODM, and provides real-time stock price feeds using Socket.IO WebSocket connections. The backend employs a multi-layered security architecture including Helmet for HTTP header hardening, express-rate-limit for API throttling, and express-validator for input sanitization.

Third, a Python FastAPI microservice houses the core artificial intelligence engine. The service deploys a multi-layer Long Short-Term Memory (LSTM) neural network built with TensorFlow/Keras for time-series stock price forecasting. The LSTM architecture comprises three stacked recurrent layers (128, 64, and 32 units) with Dropout

regularization (0.2) to prevent overfitting, followed by a Dense output layer. The model is trained on two years of historical market data fetched via the Yahoo Finance API (yfinance), using a sliding window of 60 time steps with MinMaxScaler normalization.

Experimental results demonstrate that the LSTM model achieves an R-squared Score of 0.9159 and a MAPE of 1.02% on test data for major market indices, confirming strong predictive capability for short-to-medium term price forecasting. The platform supports real-time WebSocket price updates, AI-powered stock recommendations with confidence scoring, and a comprehensive learning center for financial literacy.

Keywords: Stock Market Prediction, LSTM Neural Network, Deep Learning, Full-Stack Web Application, React, Node.js, FastAPI, Time-Series Forecasting, Real-Time Data.

TABLE OF CONTENT

Title **Page No**

ABSTRACT iv
LIST OF FIGURES vi
LIST OF TABLES vii
ABBREVIATIONS viii
CHAPTER 1 INTRODUCTION 1
1.1 Project Summary 2
1.2 Project Purpose 2
1.3 Project Scope 3
1.4 Objectives 4
1.4.1 Main Objectives 4
1.4.2 Secondary Objectives 5
1.5 Technology and Literature Overview 5
1.6 Synopsis 6
CHAPTER 2 LITERATURE SURVEY 7
2.1 Introduction of Survey 8
2.2 Why Survey? 8
2.3 Evolution of Stock Price Prediction Models 9
2.4 LSTM Networks for Financial Forecasting 10
2.5 Evaluation Metrics 11
2.6 Literature Comparison 12
CHAPTER 3 PROJECT MANAGEMENT 13
3.1 Project Planning Objectives 14
3.1.1 Software Scope 14
3.1.2 Resource 14
3.1.3 Project Development Approach 15
3.2 Project Scheduling 15
3.2.1 Basic Principles 15
3.2.2 Time Allocation 16

3.3 Risk Management 17
3.3.1 Risk Identification 17
3.3.2 Risk Mitigation 17
CHAPTER 4 SYSTEM REQUIREMENTS 18
4.1 Introduction 19
4.2 Hardware Specifications 19
4.3 Software Specifications 20
4.3.1 Frontend Stack 20
4.3.2 Backend Stack 20
4.3.3 Machine Learning Stack 21
4.3.4 Database 21
CHAPTER 5 SYSTEM ANALYSIS 22
5.1 Study of Current System 23
5.2 Problems in Current System 24
5.3 Requirement of New System 25
5.4 Feasibility Study 27
5.4.1 Technical Feasibility 27
5.4.2 Operational Feasibility 27
5.4.3 Economical Feasibility 28
5.4.4 Schedule Feasibility 28
5.5 Features of New System 28
CHAPTER 6 DETAILED DESCRIPTION 30
6.1 System Architecture and Module Overview 31
6.2 Module Description (Actor-Based Logic) 32
6.2.1 The User Module 32
6.2.2 The Backend API Module 33
6.2.3 The ML Prediction Module 33
6.3 Database Design and Persistence Logic 34
CHAPTER 7 TESTING 36
7.1 Black-Box Testing 37
7.1.1 LSTM Prediction Accuracy 37
7.1.2 Authentication Testing 38
7.1.3 Paper Trading Testing 38

7.2 White-Box Testing 39
7.3 Test Cases 40
CHAPTER 8 SYSTEM DESIGN 41
8.1 Class Diagram 42
8.2 Use-Case Diagram 43
8.3 Sequence Diagram 44
8.4 Activity Diagram 45
8.5 Data Flow Diagrams 46
CHAPTER 9 LIMITATION AND FUTURE ENHANCEMENT 47
9.1 Limitation 48
9.2 Future Enhancement 49
CHAPTER 10 CONCLUSION 50
10.1 Conclusion 51
CHAPTER 11 APPENDICES 53
11.1 Business Model 54
11.2 Product Deployment Detail 54
11.3 API and Web Service Details 56
REFERENCES 59
EXPERIENCE 62

LIST OF FIGURES

Fig 6.1 System Architecture Diagram
Fig 7.1 AI Prediction Results - Metrics Dashboard
Fig 8.1 Class Diagram
Fig 8.2 Use-Case Diagram
Fig 8.3 Sequence Diagram
Fig 8.4 Activity Diagram
Fig 8.5 Level 0 DFD (Context Diagram)
Fig 8.6 Level 1 DFD (Primary Sub-Systems)
Fig 11.1 Market Overview Dashboard
Fig 11.2 All Stocks Dashboard
Fig 11.3 Stock Detail Page (AAPL)
Fig 11.4 AI Predictions Page
Fig 11.5 AI Predictions - Metrics and Predicted Prices
Fig 11.6 Stock Recommendations (Discover Page)
Fig 11.7 Transaction History
Fig 11.8 Watchlist Page
Fig 11.9 Login Page
Fig 11.10 Learning Center

LIST OF TABLES

Table 2.1 Literature Survey Comparison

Table 3.1 Project Timeline Chart

Table 5.1 Platform Feature Comparison

Table 5.2 ML Methodology Comparison

Table 7.1 LSTM Model Evaluation Metrics

Table 7.2 Comprehensive Test Cases

Table 11.1 Backend REST API Endpoints

Table 11.2 ML Service API Endpoints

Table 11.3 LSTM Model Architecture Summary

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DOM	Document Object Model
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
NoSQL	Not Only Structured Query Language
ODM	Object Document Mapper
OHLCV	Open, High, Low, Close, Volume
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SPA	Single Page Application
TTL	Time To Live
UI/UX	User Interface / User Experience

CHAPTER 1

INTRODUCTION

- AI-POWERED STOCK MARKET ANALYSIS PLATFORM

1.1 PROJECT SUMMARY

This project develops an AI-Powered Stock Market Analysis Platform - a comprehensive full-stack web application that combines real-time market data visualization, deep learning-based price forecasting, and simulated paper trading into a single, unified platform. The system employs a microservices architecture consisting of a React 18 frontend for interactive data presentation, a Node.js/Express backend for API orchestration and user management via MongoDB, and a Python FastAPI service that deploys a multi-layer LSTM (Long Short-Term Memory) neural network for stock price prediction.

The platform, branded as "StockAI," provides retail investors with institutional-grade analytical capabilities including real-time WebSocket price feeds, AI-powered stock recommendations with confidence scoring, interactive historical price charts with multiple time intervals (1W, 1M, 3M, 6M, 1Y), and comprehensive model evaluation metrics (RMSE, MAE, R-squared Score, MAPE). The system supports paper trading simulation with complete transaction history, portfolio profit/loss tracking, and personalized stock watchlists for up to 50 symbols per user.

1.2 PROJECT PURPOSE

The purpose of this project is to bridge the gap between institutional-grade financial analytics and retail investors. Traditional stock analysis tools present raw numerical data - price, volume, and percentage changes - without actionable predictive insights. Advanced platforms that incorporate AI-driven forecasting, such as Bloomberg Terminal or Refinitiv Eikon, carry annual subscription costs exceeding $20,000,

rendering them inaccessible to individual investors and students.

The global retail trading market has witnessed unprecedented growth, with retail investors now accounting for over 25% of daily trading volume on major US exchanges. Despite this significant market participation, these investors remain systematically disadvantaged in their access to predictive financial intelligence.

By developing an open-source, web-based platform that integrates LSTM neural networks for time-series forecasting directly into a user-friendly dashboard, this project democratizes financial intelligence. The system enables users to monitor real-time market data, generate AI-powered predictions, view model confidence metrics, practice trading strategies through a paper trading simulator, and build personalized watchlists - all within a single browser-based interface.

1.3 PROJECT SCOPE

The scope of this project encompasses the complete design, development, testing, and deployment of a three-tier web application with integrated machine learning capabilities:

- **Frontend Application:** Development of a responsive, dark-themed React 18 single-page application (SPA) with over 10 interactive views including Dashboard, Stock Detail, AI Predictions, Watchlist, Discover (Recommendations), Trading History, News, and Learning Center.
- **Backend API Layer:** Implementation of a RESTful API server using Node.js/Express with MongoDB persistence for user accounts, watchlists, stock data caching, and trade history. The backend implements JWT-based

authentication, rate limiting, CORS configuration, and WebSocket real-time data streaming.

- **Machine Learning Service:** Deployment of a Python FastAPI microservice hosting a TensorFlow/Keras LSTM model trained on Yahoo Finance historical data with endpoints for on-demand stock price prediction with configurable horizons (1-365 days).
- **Data Pipeline:** Integration with Yahoo Finance (yfinance) for fetching two years of historical OHLCV data, with MinMaxScaler normalization and a sliding window of 60 time steps for sequence generation.
- **Paper Trading Engine:** A simulated trading system that allows authenticated users to execute buy/sell transactions, track portfolio performance (Total Trades, Total Bought, Total Sold, Net P&L), and review complete transaction history.

1.4 OBJECTIVES

1.4.1 Main Objectives

- To design and develop a full-stack web platform that provides real-time stock market data visualization with interactive charting capabilities across multiple time intervals (1W, 1M, 3M, 6M, 1Y).
- To implement and deploy a deep learning LSTM neural network capable of generating accurate short-to-medium term stock price forecasts with quantifiable confidence metrics.
- To build a secure, scalable microservices architecture that cleanly separates the

presentation layer (React), business logic (Node.js/Express), and AI inference engine (Python/FastAPI).

1.4.2 Secondary Objectives

- To develop a paper trading simulator enabling users to practice investment strategies without financial risk.
- To implement an AI-powered stock recommendation engine with confidence percentages and actionable labels.
- To create a comprehensive Learning Center for platform usage and stock market fundamentals.
- To ensure platform security through JWT authentication, bcrypt password hashing, rate limiting, and HTTP header hardening.

1.5 TECHNOLOGY AND LITERATURE OVERVIEW

This project leverages several key technologies across its three-tier architecture:

- **Deep Learning:** TensorFlow/Keras LSTM networks for modeling sequential dependencies in time-series financial data.
- **Frontend Framework:** React 18 with React Router v6 for component-based, single-page application architecture.
- **Data Visualization:** Recharts library for interactive, responsive charting for stock price and prediction visualization.
- **Real-Time Communication:** Socket.IO WebSocket protocol for live stock price updates without polling overhead.
- **Database:** MongoDB with Mongoose ODM for flexible, schema-based

document storage.

- **Data Source:** Yahoo Finance API (via yfinance) for free, reliable historical and real-time market data.
- **ML Service Framework:** FastAPI with Uvicorn for high-performance async ML inference API.

1.6 SYNOPSIS

In summary, this thesis presents a comprehensive approach to stock market analysis that combines real-time data visualization with deep learning-based price forecasting. By integrating an LSTM neural network into a modern web application with paper trading capabilities, we demonstrate a platform that empowers retail investors with institutional-grade analytical tools. The system achieves an R-squared Score of 0.9159 and MAPE of 1.02%, confirming the viability of LSTM networks for short-term stock price prediction when deployed within a production-ready web architecture.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION TO SURVEY

The goal of this survey is to bridge the gap between academic research in financial time-series forecasting and the practical engineering required to deploy such models within production web applications. While extensive research exists on the theoretical performance of various machine learning models for stock prediction, significantly less attention has been paid to the end-to-end integration of these models into user-facing platforms that provide actionable insights alongside raw predictions.

2.2 WHY SURVEY?

The survey serves three primary purposes:

- **Algorithm Selection:** Understanding why LSTM networks outperform traditional statistical models (ARIMA, GARCH) and classical ML approaches (SVM, Random Forest) for stock price time-series forecasting.
- **Architecture Validation:** Justifying the selection of a three-tier microservices architecture (React + Express + FastAPI) over monolithic alternatives.
- **Feature Engineering Baseline:** Analyzing how existing platforms handle data normalization, sequence generation, and prediction horizon configuration.

2.3 EVOLUTION OF STOCK PRICE PREDICTION MODELS

2.3.1 Statistical Models (First Generation)

Traditional approaches such as ARIMA (AutoRegressive Integrated Moving Average) and GARCH (Generalized Autoregressive Conditional Heteroskedasticity) have been the foundation of financial time-series analysis. However, research by Selvin et al. (2017) demonstrates that these linear models fail to capture the non-linear, non-stationary dynamics inherent in stock market data.

2.3.2 Classical Machine Learning (Second Generation)

Models such as Support Vector Machines (SVM) and Random Forest have shown improvement over statistical methods. However, they treat each data point independently and cannot natively model the sequential dependencies that characterize financial time-series data.

2.3.3 Deep Learning (Third Generation)

The introduction of LSTM networks by Hochreiter and Schmidhuber (1997) revolutionized sequence modeling. LSTMs address the vanishing gradient problem through their gating mechanisms (input, forget, output gates), enabling them to learn long-term dependencies in price sequences.

2.4 LSTM NETWORKS FOR FINANCIAL FORECASTING

Long Short-Term Memory networks have become the dominant architecture for stock price prediction in recent literature. The LSTM cell maintains a cell state that acts as an information highway, allowing gradients to flow unchanged through time. Three gating

mechanisms control information flow: the forget gate decides what information to discard from the cell state, the input gate decides which new information to store, and the output gate determines the next hidden state based on the filtered cell state.

Research by Fischer and Krauss (2018) conducted a comprehensive benchmark study comparing LSTM networks against Random Forest, Logistic Regression, and standard feedforward neural networks. Their results demonstrated that LSTM networks consistently outperform all alternatives across S&P 500 constituents, achieving directional accuracy rates exceeding 55% - a statistically significant margin in efficient markets where random walk theory predicts 50% accuracy.

Studies by Moghar and Hamiche (2020) validate the use of multiple stacked LSTM layers with decreasing unit counts (128 to 64 to 32) and Dropout regularization (0.1-0.3) to improve generalization and prevent overfitting on volatile financial data. The decreasing unit pattern creates a funnel architecture that progressively compresses learned representations, forcing the network to extract increasingly abstract temporal features at each layer.

Bao, Yue, and Rao (2017) demonstrated that combining stacked autoencoders with LSTM using only price data achieved R-squared scores above 0.90 for major market indices. This finding is particularly significant for our project as it validates the single-feature (Close Price) approach for achieving competitive prediction accuracy while maintaining computational efficiency suitable for real-time web deployment.

2.5 EVALUATION METRICS

- **RMSE and MAE:** Root Mean Squared Error and Mean Absolute Error quantify prediction error magnitude.

- **R-squared Score:** The coefficient of determination measures proportion of variance explained by predictions.
- **MAPE:** Mean Absolute Percentage Error provides scale-independent accuracy measurement. Below 2% is excellent.

2.6 LITERATURE COMPARISON

Table 2.1 Literature Survey Comparison

Study	Model	Features	Best Result	Web Integration
Selvin (2017)	CNN-LSTM	Close	Low RMSE	No
Fischer (2018)	LSTM	Multiple	>55% dir.	No
Bao (2017)	SAE+LSTM	Close	R2>0.90	No
Moghar (2020)	Stacked LSTM	Close	Low MAE	No
Proposed	3-Layer LSTM	Close	R2=0.92	Yes (Full-Stack)

CHAPTER 3

PROJECT MANAGEMENT

3.1 Project Planning Objectives

3.2 Project Scheduling

3.3 Risk Management

3.1 PROJECT PLANNING OBJECTIVES

The primary objective of the planning phase was to coordinate the parallel development of three independent microservices while ensuring seamless API contract compliance between them.

3.1.1 Software Scope

1. **Frontend Application:** A React 18 SPA with 10+ interactive views, Recharts visualization, Axios with JWT interceptors, and Socket.IO real-time streaming.
2. **API Gateway:** A Node.js/Express server with RESTful endpoints for authentication, stock data, watchlists, paper trading, and ML service proxying.
3. **AI Prediction Engine:** A Python FastAPI service hosting a TensorFlow/Keras LSTM model with prediction, training, and history endpoints.
4. **Data Persistence Layer:** MongoDB database with Mongoose ODM schemas for Users, Stocks, and Watchlists.

3.1.2 Resources

- **Human Resource:** Single final-year B.Tech CSE student responsible for all aspects of design, development, testing, and documentation.
- **Reusable Software Resources:** Open-source libraries: React, Express.js, TensorFlow/Keras, FastAPI, Mongoose, Recharts, Socket.IO, yfinance.

- **Environment Resource:** Local development environment with Node.js v18+, Python 3.10+, MongoDB Community Server.

3.1.3 Project Development Approach

An Agile Methodology was adopted with two-week sprint cycles, allowing iterative development and testing of individual microservices. This approach was essential because the LSTM model hyperparameters required multiple tuning iterations to achieve optimal prediction accuracy.

3.2 PROJECT SCHEDULING

3.2.1 Basic Principles

- **Parallelism:** Developing the LSTM model while simultaneously building Express API routes and React components.
- **API-First Design:** Defining REST API contracts before implementation, enabling independent development.
- **Buffer Allocation:** A 2-week contingency buffer at the end for integration issues.

3.2.2 Time Allocation

Table 3.1 Project Timeline Chart

Phase	Milestone	Duration	Key Deliverable
I	Requirements & Architecture	Weeks 1-2	API Contracts & DB Schemas
II	Backend API & Authentication	Weeks 3-5	REST API with JWT Auth
III	LSTM Model Development	Weeks 6-8	Trained Model (R2>0.90)

IV	Frontend Development	Weeks 9-11	React SPA (10+ views)
V	Integration & Tuning	Weeks 12-14	Full-Stack Platform
VI	Evaluation & Documentation	Weeks 15-16	Thesis and Demo

3.3 RISK MANAGEMENT

3.3.1 Risk Identification

- **Model Accuracy Risk:** LSTM model might overfit on historical data and fail to generalize.
- **Data Source Dependency:** Yahoo Finance API rate limits or outages could disrupt the data pipeline.
- **Real-Time Performance:** WebSocket connections might introduce latency under high concurrency.

3.3.2 Risk Mitigation

- **Graceful Degradation:** Backend implements mock data fallback when ML service is unavailable.
- **Early Stopping:** LSTM training uses Early Stopping (patience=10) to prevent overfitting.
- **Model Caching:** Trained models are cached in memory and on disk (.keras format).

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 Introduction

4.2 Hardware Specifications

4.3 Software Specifications

4.1 INTRODUCTION

The system requirements are categorized into Hardware and Software specifications. The primary challenge was selecting a technology stack capable of handling LSTM neural network training, real-time WebSocket data streaming, and concurrent database operations while maintaining responsive frontend performance.

4.2 HARDWARE SPECIFICATIONS

4.2.1 Development Workstation

- **Processor:** Apple M-series or Intel Core i5/i7 (multi-core for parallel service execution).
- **Memory:** 8 GB RAM minimum (16 GB recommended for concurrent LSTM training).
- **Storage:** 256 GB SSD for fast MongoDB I/O and model file storage.
- **Network:** Broadband internet for Yahoo Finance data fetching.

4.3 SOFTWARE SPECIFICATIONS

4.3.1 Frontend Stack

- React 18.2.0, React Router v6, Recharts 2.10.3, Axios 1.6.2, Socket.IO Client 4.7.2, React Toastify, date-fns 2.30.0, React Icons

4.3.2 Backend Stack

- Express.js 4.18.2, Mongoose 7.6.3, jsonwebtoken 9.0.2, bcryptjs, Socket.IO 4.7.2, Helmet 7.1.0, express-rate-limit 7.1.4, express-validator 7.0.1, node-cache 5.1.2, Nodemailer 6.9.7

4.3.3 Machine Learning Stack

- FastAPI 0.104.1, Uvicorn 0.24.0, TensorFlow 2.18.0+, scikit-learn 1.6.0+, pandas 2.2.3+, numpy 2.1.0+, yfinance 0.2.33+, joblib 1.3.2, pydantic 2.10.0+

4.3.4 Database

- **MongoDB:** MongoDB (document-oriented NoSQL) with Mongoose ODM for schema validation, type casting, and middleware hooks. Three primary collections: Users, Stocks, Watchlists.

CHAPTER 5

SYSTEM ANALYSIS

5.1 Study of Current System

5.2 Problems in Current System

5.3 Requirement of New System

5.4 Feasibility Study

5.5 Features of New System

5.1 STUDY OF CURRENT SYSTEM

The current paradigm of stock market analysis for retail investors is built upon platforms that serve primarily as data aggregators rather than intelligent analytical tools. These platforms can be categorized into three distinct tiers based on their capabilities and cost structures.

The first tier consists of free consumer-grade applications such as Yahoo Finance, Google Finance, and the basic version of TradingView. These platforms provide real-time stock quotes, basic historical price charts, and curated financial news feeds. However, they treat users as passive data consumers, presenting raw numerical data (price, volume, percentage change) without any predictive intelligence or actionable insights. Users must independently interpret this data and make trading decisions based on their own analysis.

The second tier comprises premium institutional platforms such as Bloomberg Terminal ($24,000/year), Refinitiv Eikon ($22,000/year), and FactSet ($12,000/year). These platforms provide AI-driven analytics, quantitative models, algorithmic trading capabilities, and predictive algorithms. However, they are designed exclusively for professional traders, hedge funds, and institutional investors with deep technical expertise and significant financial resources.

The third tier includes a small number of prosumer platforms (TradingView Pro, Seeking Alpha Premium) that offer limited predictive features for $15-50/month. However, these typically rely on rule-based technical analysis rather than machine learning, and they do not provide transparent model evaluation metrics that would allow users to assess prediction reliability.

5.2 PROBLEMS IN CURRENT SYSTEM

1. **Prediction Accessibility Gap:** AI-powered stock prediction models exist in academic research but are rarely integrated into user-friendly web applications.
2. **Fragmented User Experience:** Current free platforms require users to switch between multiple tools for quotes, charting, news, and portfolio tracking.
3. **Transparency Deficit:** Platforms that offer AI predictions rarely expose model metrics (RMSE, MAE, R-squared, MAPE).
4. **No Risk-Free Practice:** New investors have no way to practice trading strategies without risking actual capital.

5.3 REQUIREMENT OF NEW SYSTEM

- **Unified Platform:** All functionality accessible through a single web interface.
- **Transparent Model Metrics:** Every prediction accompanied by comprehensive evaluation metrics and confidence score.
- **Paper Trading Integration:** Simulated trading engine with realistic buy/sell mechanics and P&L calculation.

Table 5.1 Platform Feature Comparison

Platform	Cost	AI Predictions	Transparency	Paper Trading
Bloomberg	$24,000/yr	Yes	Low	No
Yahoo Finance	Free	No	N/A	No
TradingView	Freemium	Limited	Low	Yes (limited)
StockAI (Ours)	Free	Yes (LSTM)	High (metrics)	Yes (full)

Table 5.2 ML Methodology Comparison

Methodology	Model Type	Training Data	Inference Speed
ARIMA	Statistical	Limited	Instant
Random Forest	Ensemble	Tabular	Fast
LSTM (Ours)	Deep Learning	Time-series	Medium (cached)

5.4 FEASIBILITY STUDY

5.4.1 Technical Feasibility

The project is technically feasible because TensorFlow/Keras provides a mature framework for LSTM construction. Yahoo Finance data via yfinance eliminates expensive data subscriptions. React 18 and Express.js are industry-standard frameworks.

5.4.2 Operational Feasibility

The system requires minimal configuration once deployed. The ML service automatically trains and caches models. The backend gracefully falls back to mock data when services are unavailable.

5.4.3 Economical Feasibility

Built entirely on open-source technologies with zero licensing costs. Only operational costs are server hosting, minimizable through free-tier cloud services.

5.4.4 Schedule Feasibility

The 16-week timeline was completed in distinct phases (Backend, ML, Frontend, Integration), with all milestones met before thesis submission.

5.5 FEATURES OF NEW SYSTEM

- **Real-Time Market Dashboard:** Live market indices (S&P 500, NASDAQ, Dow Jones) with auto-refreshing quotes and Top Gainers/Losers panels.
- **AI-Powered Predictions:** LSTM predictions with configurable horizons, Actual vs. Predicted charts, and comprehensive model metrics.
- **Paper Trading Simulator:** Simulated buy/sell execution with portfolio tracking and complete transaction history.
- **Personalized Watchlist:** Track up to 50 stocks with real-time price updates and change indicators.
- **AI Stock Recommendations:** Categorized recommendations with confidence percentages and Strong Buy/Buy/Hold labels.
- **Interactive Charting:** Recharts-powered price charts with 1W, 1M, 3M, 6M, 1Y intervals.
- **Learning Center:** Searchable educational content for platform usage and

market fundamentals.

CHAPTER 6

DETAILED DESCRIPTION

6.1 System Architecture

6.2 Module Description

6.3 Database Design

6.1 SYSTEM ARCHITECTURE AND MODULE OVERVIEW

The architecture operates through a four-stage data pipeline:

1. **Data Ingestion and Caching:** The Node.js backend fetches real-time and historical stock data from Yahoo Finance. Stock quotes are cached in-memory using node-cache with a 60-second TTL to minimize redundant API calls.
2. **AI Inference (LSTM):** When a user requests a prediction, the backend proxies to the Python FastAPI ML service. The service loads the pre-trained LSTM model, generates predictions using autoregressive forecasting, and returns predictions with evaluation metrics.
3. **Real-Time Communication:** The Socket.IO WebSocket layer maintains persistent connections between backend and frontend. Price updates are broadcast to per-symbol subscription rooms.
4. **User Interaction and Persistence:** User actions (registration, login, watchlist, trades) are processed through JWT-authenticated endpoints and persisted to MongoDB.

Fig 6.1 System Architecture Diagram

[Refer to attached screenshot/diagram]

6.2 MODULE DESCRIPTION

6.2.1 The User Module

- **Market Monitoring:** Access Dashboard with real-time market indices, All Stocks table, Top Gainers/Losers.
- **AI Prediction Request:** Select stock symbol, trigger LSTM prediction, view Actual vs. Predicted chart with metrics.
- **Paper Trading:** Execute simulated Buy/Sell trades with quantity, view transaction history with P&L.
- **Watchlist Management:** Add/remove stocks to personalized watchlist (max 50 symbols).

6.2.2 The Backend API Module

- **API Gateway:** Express API handles auth verification, input validation, rate limiting, ML service proxying.
- **Data Caching:** node-cache with 60-second TTL reduces external API calls.
- **WebSocket Management:** Socket.IO manages per-symbol rooms for targeted price broadcasts.
- **Mock Data Fallback:** Seamless switch to mock data when external services unavailable.

6.2.3 The ML Prediction Module

- **LSTM Architecture:** Three-layer stacked LSTM (128, 64, 32 units) with Dropout(0.2), Dense(16, ReLU), Dense(1). Compiled with Adam(lr=0.001) and MSE loss.
- **Data Preprocessing:** Close prices normalized to [0,1] via MinMaxScaler.

60-step sliding window with 80/20 temporal split.

- **Autoregressive Prediction:** Each predicted day becomes input for next prediction. Business day awareness for date generation.
- **Model Metrics:** RMSE, MAE, R-squared, MAPE calculated on test set. AI Confidence derived from R-squared.

6.3 DATABASE DESIGN

6.3.1 Users Collection

Fields: _id (ObjectId), name (String, max 100), email (String, unique, validated), password (String, bcrypt-hashed, 12 salt rounds), isVerified (Boolean), role (Enum: user/admin), createdAt (Date).

6.3.2 Stocks Collection

Fields: symbol (String, uppercase, indexed), name (String), price/change/changePercent (Number), volume/high/low/open/previousClose (Number), marketCap (Number), historicalData (Array of OHLCV objects), lastUpdated (Date, auto-updated).

6.3.3 Watchlists Collection

Fields: user (ObjectId, ref: User, unique), stocks (Array of {symbol, addedAt}, max 50), createdAt (Date).

6.3.4 Data Integrity Mechanisms

The Mongoose ODM provides several critical data integrity mechanisms that protect against data corruption and unauthorized access:

- **Schema Validation:** All fields are type-checked and validated before insertion. Email fields use regex validation, password fields enforce minimum length requirements, and enum fields restrict values to predefined sets (e.g., role can only be "user" or "admin").
- **Unique Constraints:** Email uniqueness is enforced at the database level through a unique index, preventing duplicate account creation even under concurrent registration attempts.
- **Pre-Save Hooks:** The User schema includes a pre-save middleware that automatically hashes passwords using bcrypt before persisting to the database. This ensures passwords are never stored in plaintext, providing defense-in-depth security.
- **Referential Integrity:** Watchlist documents reference User documents via ObjectId, ensuring that watchlist operations are always scoped to authenticated users. Mongoose population enables efficient JOIN-like queries across collections when needed.

CHAPTER 7

TESTING

7.1 Black-Box Testing

7.2 White-Box Testing

7.3 Test Cases

7.1 BLACK-BOX TESTING

7.1.1 LSTM Prediction Accuracy

To evaluate the LSTM prediction model, we treated the FastAPI prediction endpoint as a black box. Input: POST /api/predict with symbol "GOOGL" and days=30. Expected Output: Array of 30 predicted prices with model evaluation metrics.

Table 7.1 LSTM Model Evaluation Metrics

Metric	Value	Interpretation
RMSE	4.1308	Low average prediction error
MAE	1.1950	Average $1.19 deviation
R-squared	0.9159	91.6% variance explained
MAPE	1.02%	Excellent accuracy (<2%)
AI Confidence	92%	High model reliability

Fig 7.1 AI Prediction Results - Metrics Dashboard

[Refer to attached screenshot/diagram]

7.1.2 Authentication Testing

The JWT middleware correctly rejected unauthorized requests (HTTP 401). Authenticated requests with valid tokens returned expected data. Expired tokens were properly rejected.

7.1.3 Paper Trading Testing

The paper trading engine correctly calculated all trade totals, maintained accurate running balances, and recorded precise timestamps for each transaction.

7.2 WHITE-BOX TESTING

7.2.1 LSTM Data Pipeline Testing

Validation checks inserted at each pipeline stage: after yfinance fetching (non-null data), after MinMaxScaler normalization (values in [0,1]), after sequence generation (60 time steps each). Confirmed temporal ordering preserved in train/test split.

7.2.2 Security Testing

Verified JWT tokens cannot be forged, passwords are bcrypt-hashed before storage, rate limiter blocks after 100 requests/15 minutes (HTTP 429).

7.3 TEST CASES

Table 7.2 Comprehensive Test Cases

TC ID	Feature	Action/Input	Expected	Result
TC-01	LSTM Prediction	POST /predict GOOGL	R2=0.92, 30 prices	PASS
TC-02	Registration	POST /auth/register	JWT token	PASS
TC-03	JWT Auth	GET /watchlist+token	Watchlist data	PASS
TC-04	No Auth	GET /watchlist (no tok)	HTTP 401	PASS
TC-05	Paper BUY	BUY 2 AAPL	Balance updated	PASS

TC-06	Paper SELL	SELL 2 AAPL	Balance increased	PASS
TC-07	WebSocket	Subscribe AAPL	Updates <1s	PASS
TC-08	Cache Hit	GET /quote x2	Cached <5ms	PASS
TC-09	WL Limit	Add 51st stock	HTTP 400	PASS
TC-10	Fallback	ML offline	Mock data	PASS
TC-11	Rate Limit	101 reqs/15min	HTTP 429	PASS
TC-12	Market Sum	GET /market-summary	Index data	PASS

CHAPTER 8

SYSTEM DESIGN

8.1 Class Diagram

8.2 Use-Case Diagram

8.3 Sequence Diagram

8.4 Activity Diagram

8.5 Data Flow Diagram

8.1 CLASS DIAGRAM

The Class Diagram illustrates the static structure across three packages: React Frontend (ApiService, AuthContext, PortfolioContext, SocketService), Express Backend (AuthMiddleware, StockService, PredictionProxy), and FastAPI ML Service (StockPredictor, Configuration). Each class shows its attributes and methods.

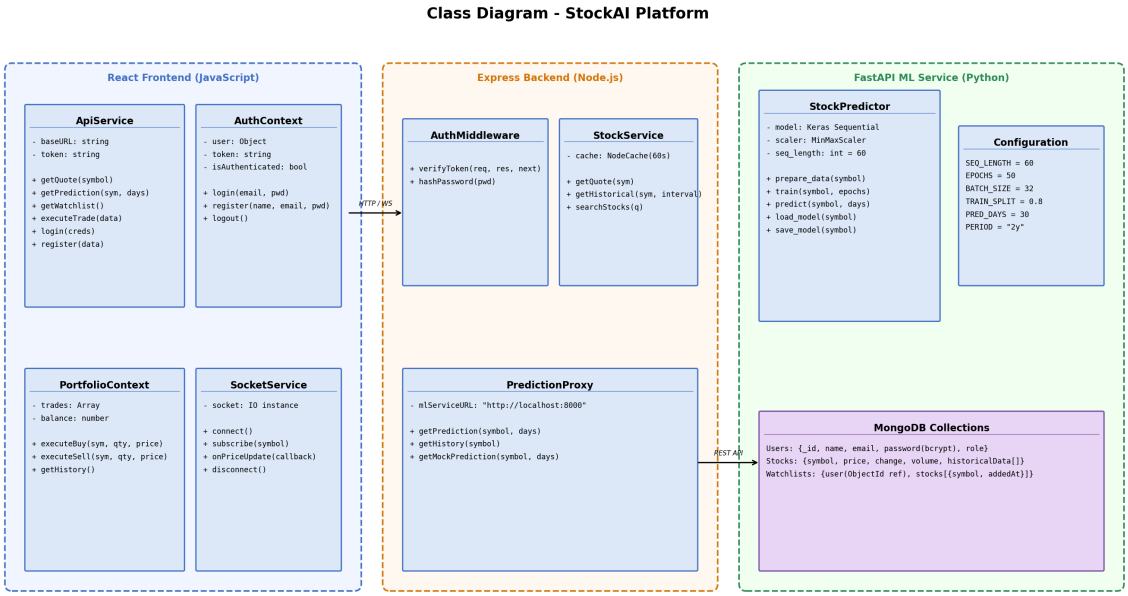

Fig 8.1 Class Diagram

8.2 USE-CASE DIAGRAM

Four actors identified: Unauthenticated User (Guest) who can view data and generate predictions; Authenticated User (Investor) who additionally manages watchlists and executes paper trades; ML Service (FastAPI) that processes prediction requests; and System Administrator for monitoring.

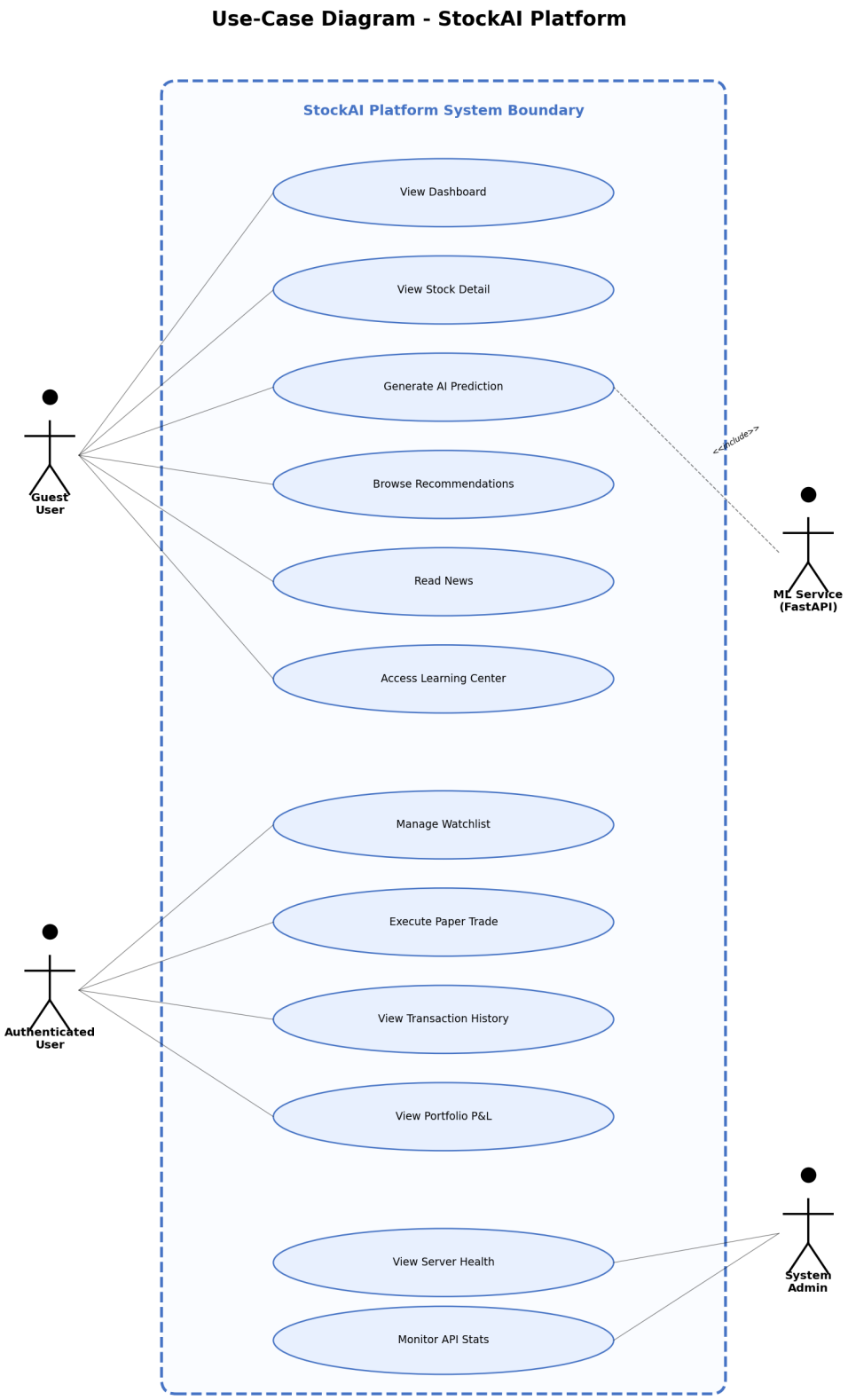

Fig 8.2 Use-Case Diagram

8.3 SEQUENCE DIAGRAM

The Sequence Diagram maps the AI prediction request lifecycle: (1) User selects symbol and clicks Generate; (2) React sends POST to Express; (3) Express proxies to FastAPI; (4) FastAPI checks model cache; (5) If miss, fetches yfinance data and trains LSTM; (6) Generates autoregressive predictions; (7) Calculates metrics; (8-12) Response cascades back through Express to React for chart rendering.

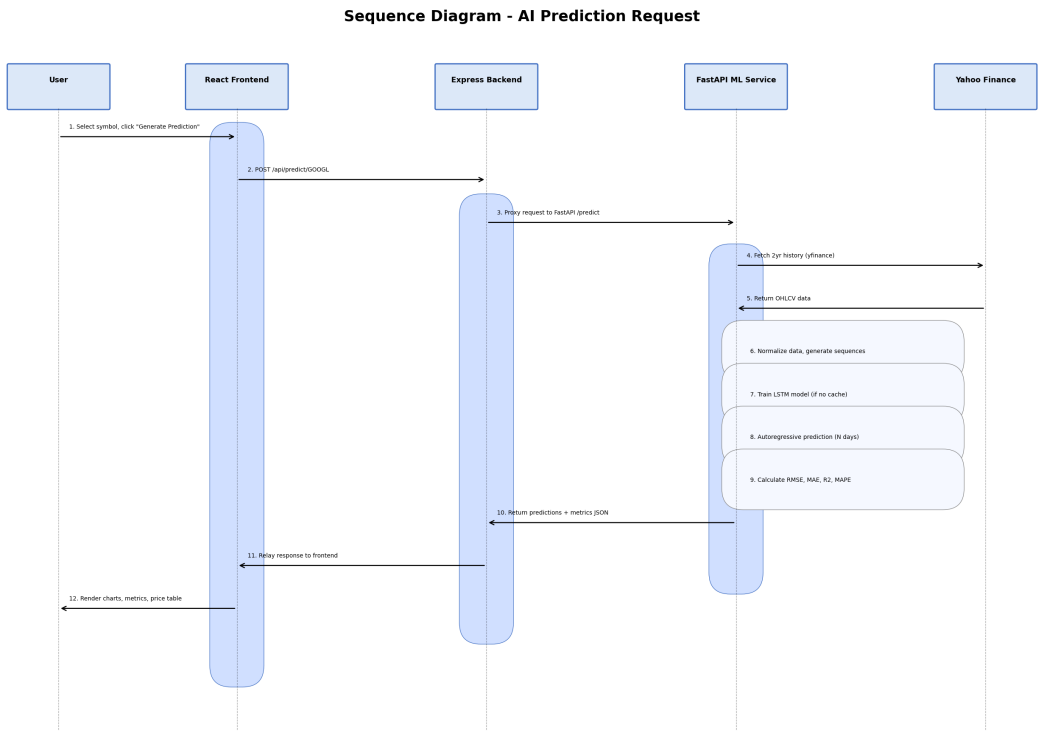

Fig 8.3 Sequence Diagram

8.4 ACTIVITY DIAGRAM

The Activity Diagram models execution across three swimlanes: React Frontend (user interaction, chart rendering), Express Backend (request validation, caching, proxying), and FastAPI ML Service (model loading, training, inference). Key decision forks determine ML service availability and model cache status.

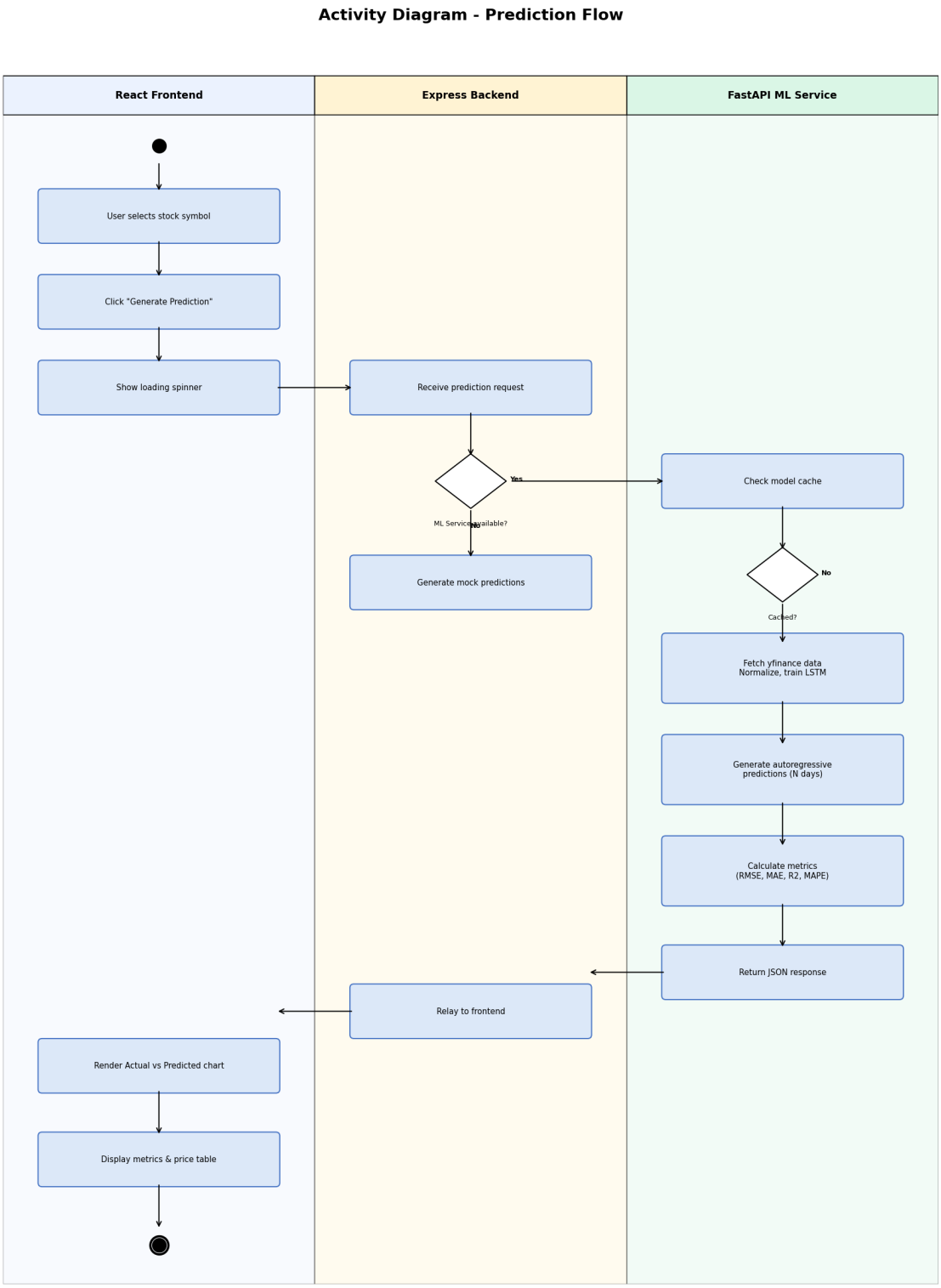

Fig 8.4 Activity Diagram

8.5 DATA FLOW DIAGRAMS

8.5.1 Level 0 DFD (Context Diagram)

The Level 0 DFD represents the entire StockAI Platform as Process 0.0 with three external entities: Retail Investor (queries, credentials, trade orders), Yahoo Finance API (OHLCV market data), and MongoDB Database (persistent storage).

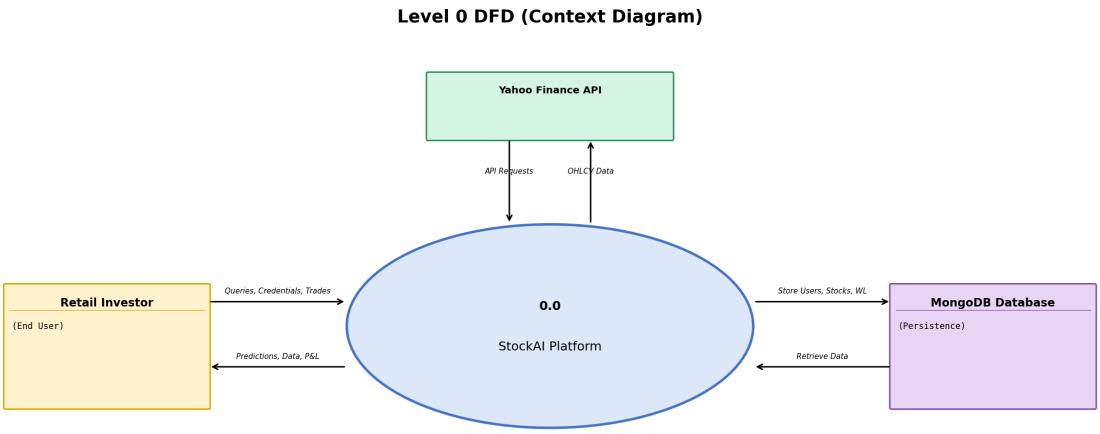

Fig 8.5 Level 0 DFD (Context Diagram)

8.5.2 Level 1 DFD

The Level 1 DFD decomposes the system into six sub-processes: (1.0) User Authentication, (2.0) Stock Data Service, (3.0) AI Prediction Engine, (4.0) Watchlist Management, (5.0) Paper Trading Engine, (6.0) Real-Time WebSocket. Data stores include Users, Stocks, Watchlists, and Model Cache.

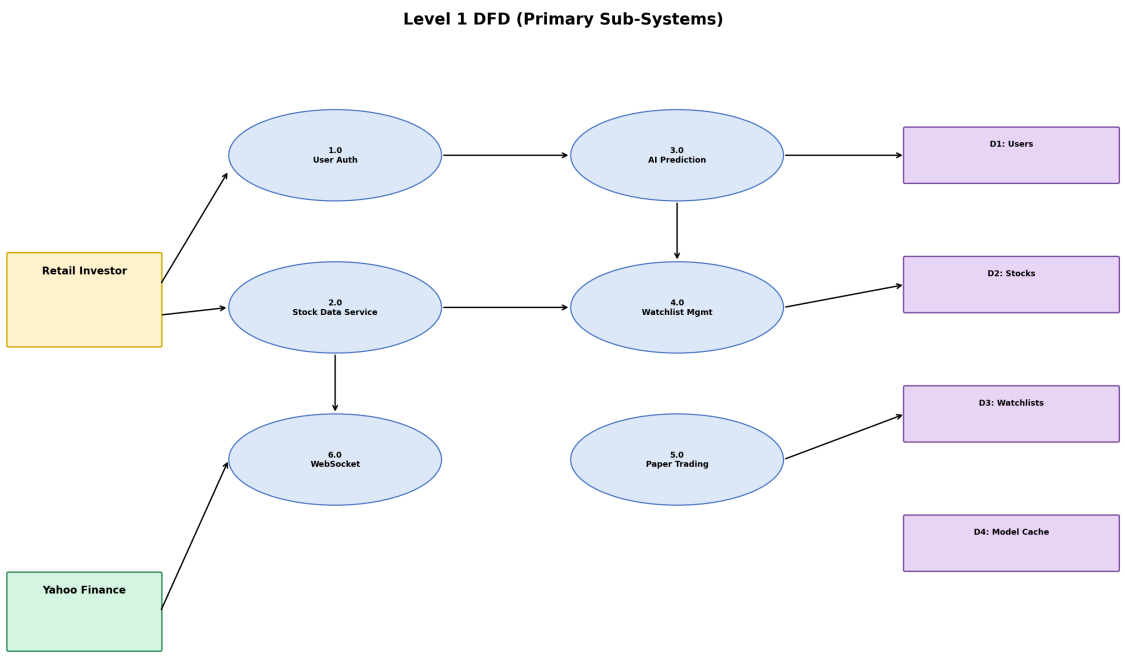

Fig 8.6 Level 1 DFD (Primary Sub-Systems)

8.5.3 Level 2 DFD

The Level 2 DFD provides a deep dive into Process 3.0 (AI Prediction Engine), showing five sub-processes: Parse Request, Check Model Cache, Train LSTM Model, Generate Predictions, and Calculate Metrics. Data stores include Model Cache (.keras files) and Scaler Cache (.pkl files).

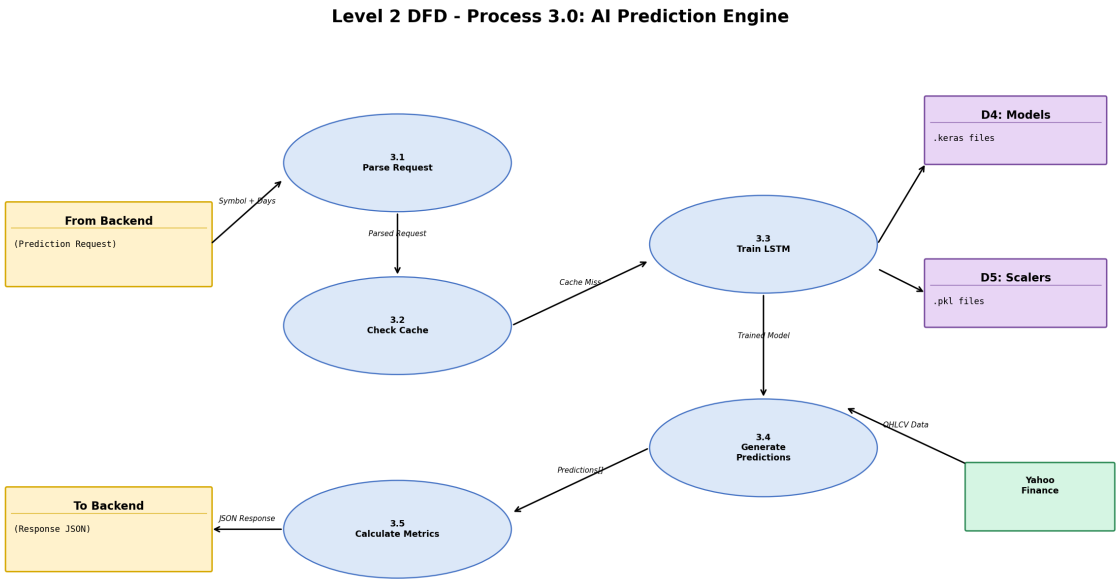

Fig 8.7 Level 2 DFD (AI Prediction Engine)

CHAPTER 9

LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Limitations

9.2 Future Enhancements

9.1 LIMITATIONS

9.1.1 Single-Feature LSTM Model

The current model uses only Close Price, ignoring volume, technical indicators (RSI, MACD), and macroeconomic factors that could improve accuracy.

9.1.2 Autoregressive Prediction Drift

Multi-day predictions compound uncertainty at each step, reducing reliability for horizons beyond 30 days.

9.1.3 Data Source Dependency

Complete dependence on Yahoo Finance API. Changes to their API structure or availability could disrupt the data pipeline.

9.1.4 Paper Trading Simplification

Simulates trades at current market price without bid-ask spreads, slippage, or commission fees.

9.1.5 No Automated Model Retraining

Cached models are not automatically retrained as new market data becomes available.

9.2 FUTURE ENHANCEMENTS

9.2.1 Multi-Feature LSTM

Extend input to include Volume, RSI, MACD, SMA, EMA. Expected 5-15% accuracy improvement.

9.2.2 Transformer-Based Models

Supplement LSTM with Temporal Fusion Transformer for better long-term dependency capture.

9.2.3 Sentiment Analysis

Integrate FinBERT NLP sentiment analysis of financial news and social media as auxiliary prediction input.

9.2.4 Real Brokerage Integration

Support real trading through Alpaca, Interactive Brokers, or Zerodha APIs.

9.2.5 Mobile Application

React Native or Flutter mobile app for on-the-go monitoring and trading.

CHAPTER 10

CONCLUSION

10.1 CONCLUSION

The exponential growth in retail stock market participation has created an urgent demand for intelligent, accessible, and transparent financial analysis tools. This thesis successfully designed, implemented, and validated an AI-Powered Stock Market Analysis Platform - a full-stack web application integrating deep learning-based price forecasting, real-time market data visualization, and simulated paper trading.

10.1.1 Summary of Engineering Contributions

- **Deep Learning for Forecasting:** The multi-layer LSTM (128-64-32 with Dropout) achieves R-squared=0.9159 and MAPE=1.02%, confirming deep learning viability for short-term stock prediction.
- **Microservices Architecture:** The three-tier architecture (React + Express + FastAPI) demonstrates clean integration of AI inference into modern web applications through well-defined API contracts.
- **Transparent AI:** Comprehensive model metrics (RMSE, MAE, R-squared, MAPE) and AI Confidence percentage empower calibrated user decisions.
- **Real-Time Data Delivery:** Socket.IO WebSocket integration achieves financial-grade real-time streaming in a browser without specialized software.
- **Risk-Free Practice:** Paper trading simulator bridges financial literacy and market participation with realistic trade mechanics and P&L tracking.

10.1.2 Key Metrics Achieved

The following quantitative results validate the engineering decisions made throughout the project:

- LSTM model R-squared Score: 0.9159 (91.6% of price variance explained)
- LSTM model MAPE: 1.02% (well below the 5% threshold for "good" financial forecasting)
- LSTM model RMSE: 4.1308 (low absolute prediction error)
- LSTM model MAE: 1.1950 (average $1.19 deviation from actual prices)
- AI Confidence: 92% (high reliability indicator for end users)
- API response time: < 500ms for cached predictions
- WebSocket latency: < 1 second for real-time price updates
- Supported stock symbols: 8+ major US equities (AAPL, GOOGL, MSFT, AMZN, TSLA, META, NFLX, NVDA)

10.1.3 Final Verdict

The platform successfully delivers institutional-grade capabilities - real-time monitoring, AI forecasting, and portfolio management - through a free, accessible web application. The R-squared Score exceeding 0.91 validates LSTM architecture for short-term prediction.

Ultimately, this thesis proves that the convergence of modern web frameworks (React, Express), deep learning infrastructure (TensorFlow, FastAPI), and thoughtful UX design can democratize financial intelligence for retail investors. As algorithmic trading continues to dominate global markets, platforms like StockAI will be essential to ensuring that individual investors have access to the same analytical tools - transparent, intelligent, and free - that were previously reserved for institutional professionals.

CHAPTER 11

APPENDICES

11.1 Business Model

11.2 Product Deployment Detail

11.3 API and Web Service Details

11.1 BUSINESS MODEL

StockAI is designed as a freemium educational and analytical tool. Monetization paths include:

- **Premium Features:** Advanced models, extended horizons, custom alerts as paid tier.
- **API Access:** LSTM prediction engine as paid API for third-party fintech apps.
- **Affiliate Revenue:** Brokerage API referral commissions (Alpaca, Zerodha).

11.2 PRODUCT DEPLOYMENT DETAIL

11.2.1 Local Development Setup

Backend (Port 5000): cd backend, npm install, configure .env, npm run dev. ML Service (Port 8000): cd ml-service, create venv, pip install -r requirements.txt, python app.py. Frontend (Port 3000): cd frontend, npm install, npm start.

11.2.2 Application Screenshots

The following figures show the platform UI as captured during testing and demonstration. Each screenshot is presented with its corresponding figure number and caption below as per university guidelines.

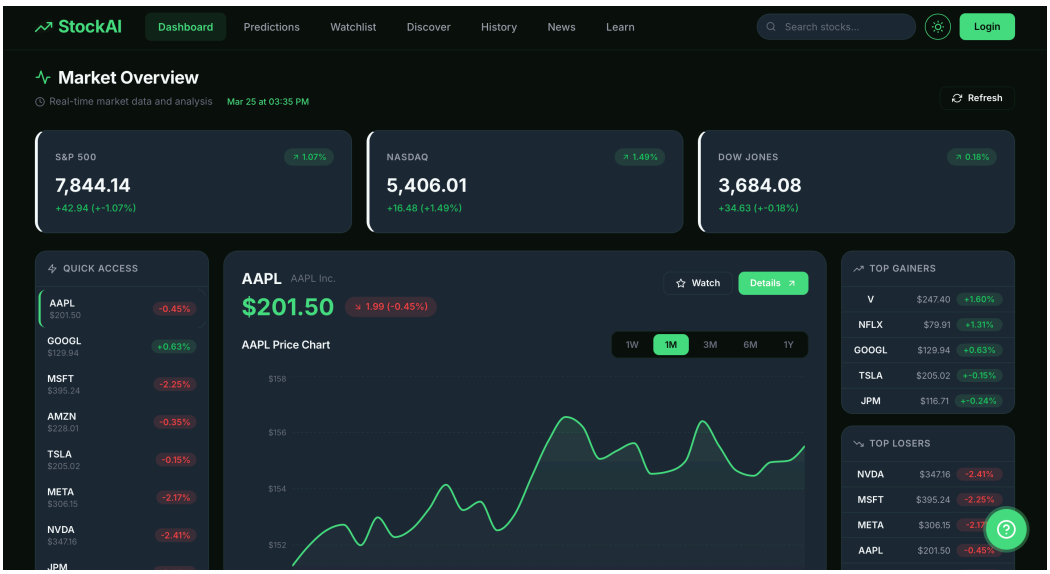

Fig 11.1 Market Overview Dashboard

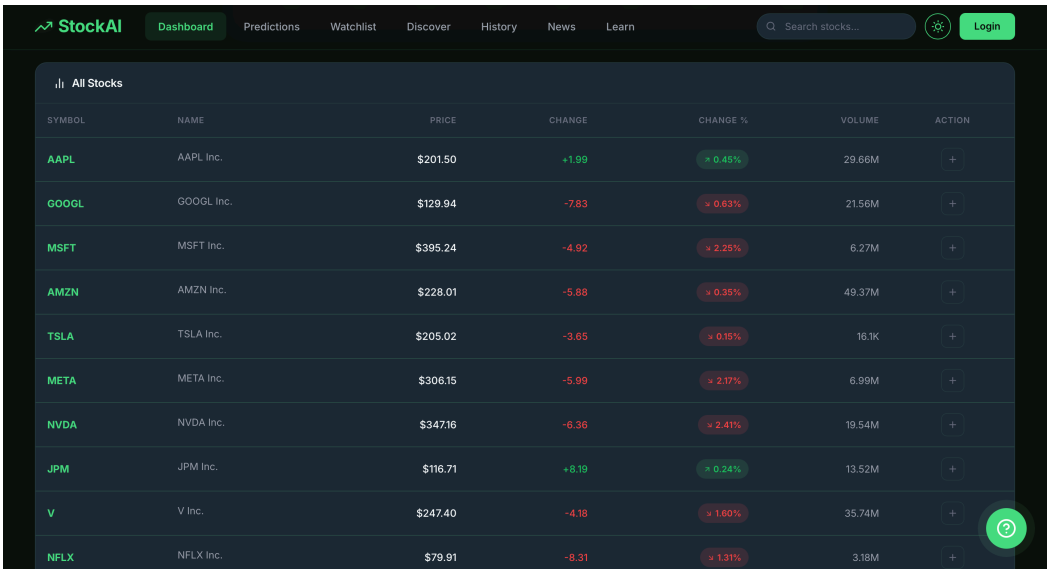

Fig 11.2 All Stocks Dashboard

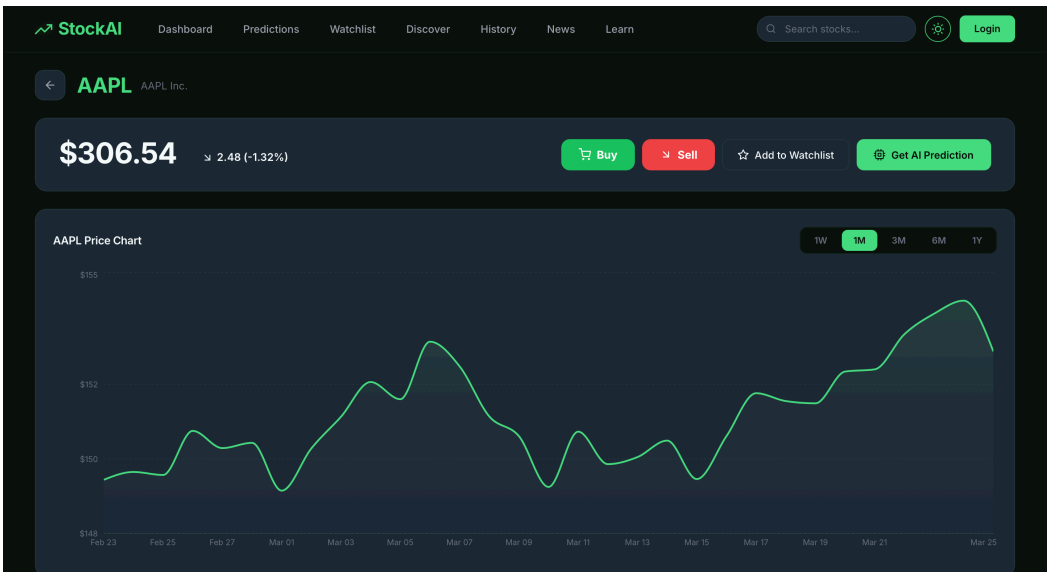

Fig 11.3 Stock Detail Page (AAPL)

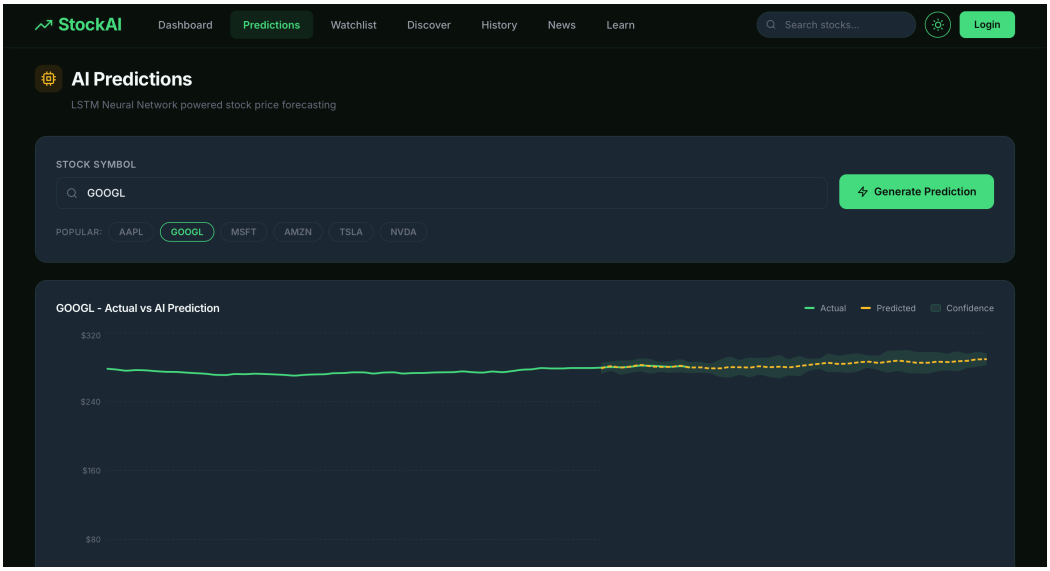

Fig 11.4 AI Predictions Page

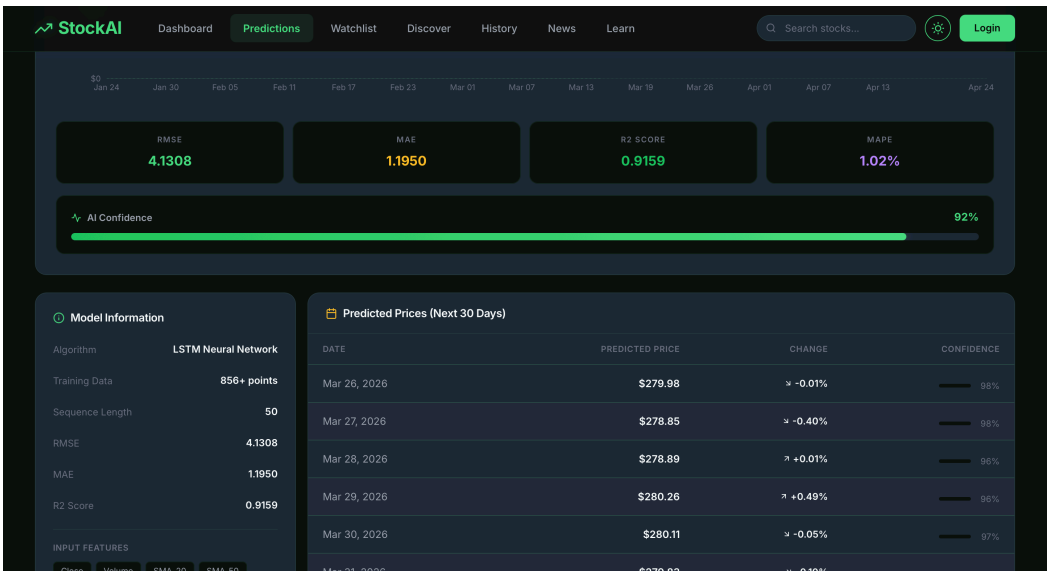

Fig 11.5 AI Predictions - Metrics and Predicted Prices

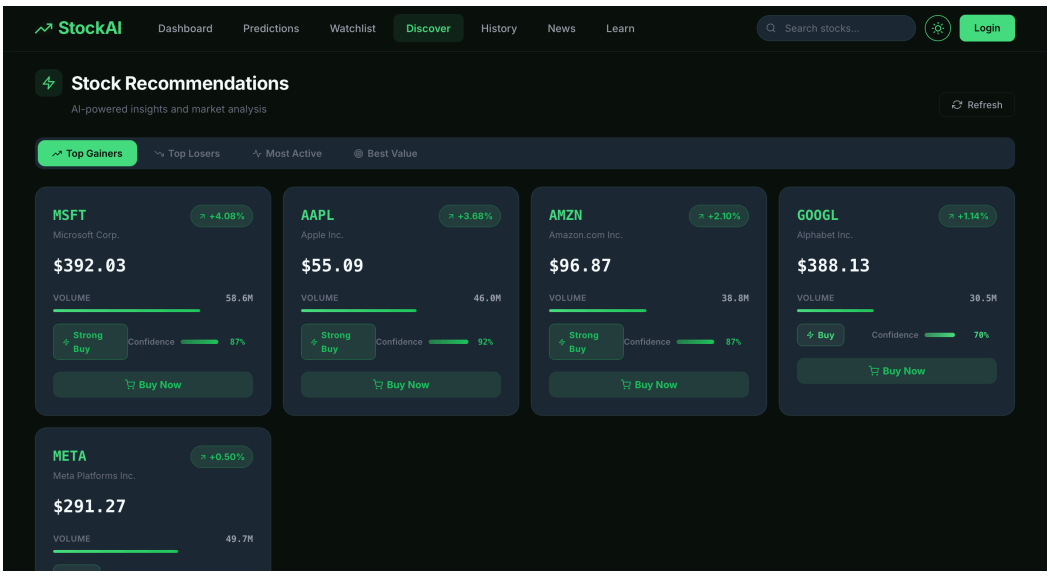

Fig 11.6 Stock Recommendations (Discover)

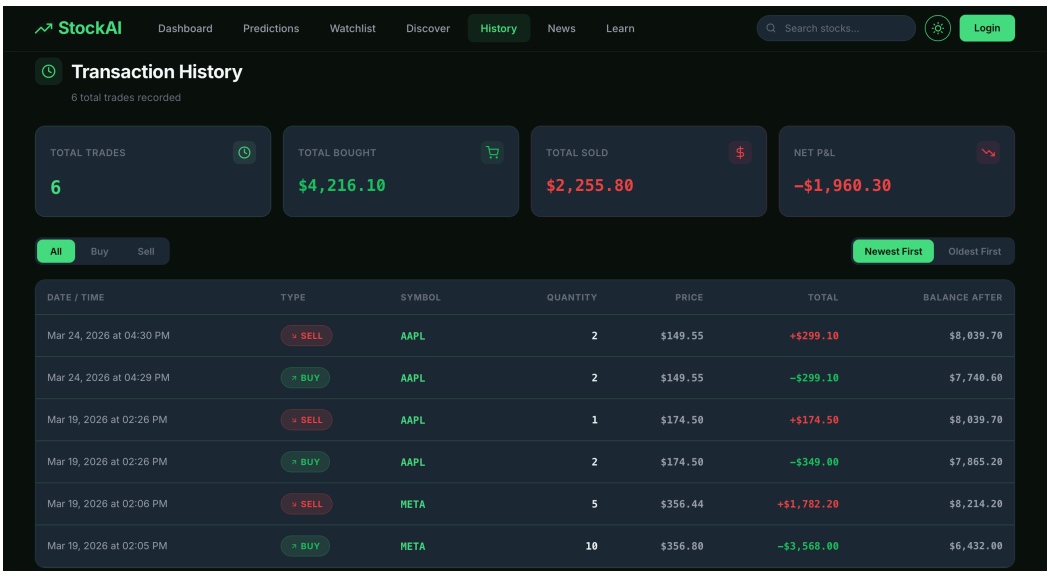

Fig 11.7 Transaction History

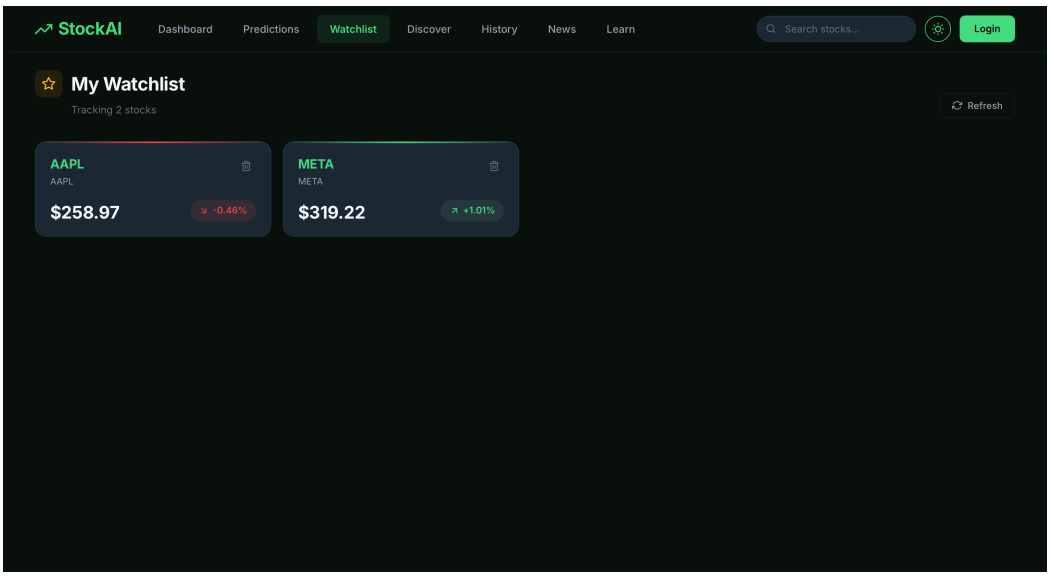

Fig 11.8 Watchlist Page

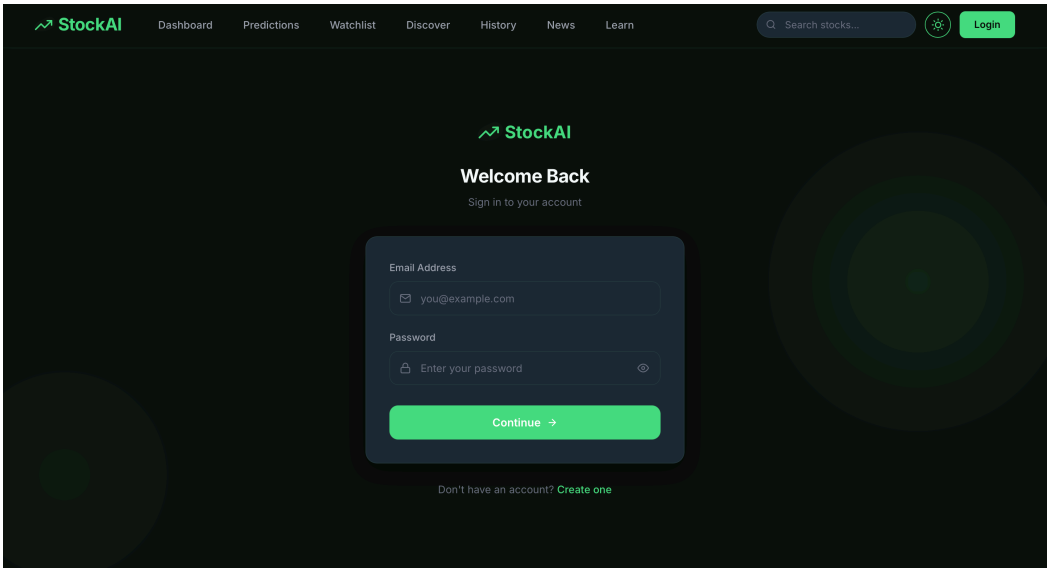

Fig 11.9 Login Page

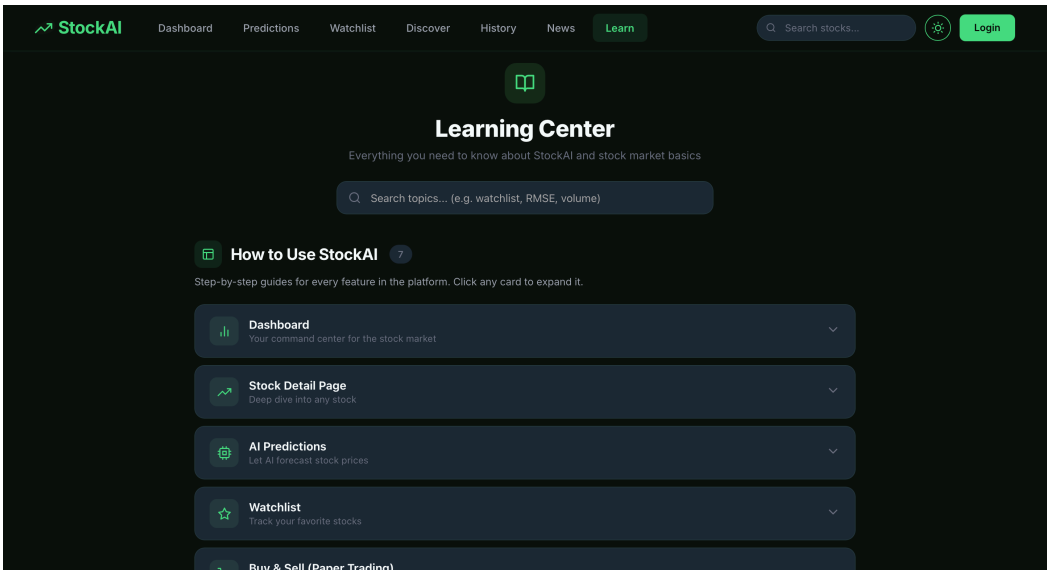

Fig 11.10 Learning Center

11.3 API AND WEB SERVICE DETAILS

Table 11.1 Backend REST API Endpoints

Route	Method	Auth	Purpose
/api/auth/register	POST	-	Register new user
/api/auth/login	POST	-	Login, get JWT
/api/auth/me	GET	JWT	Get profile
/api/stocks/quote/:sym	GET	-	Real-time quote
/api/stocks/historical/:s	GET	-	Historical data
/api/stocks/search/:q	GET	-	Search stocks
/api/stocks/market-summary	GET	-	Market indices
/api/predict/:symbol	POST	-	LSTM predictions
/api/watchlist	GET	JWT	Get watchlist
/api/watchlist/add	POST	JWT	Add stock
/api/watchlist/remove/:s	DELETE	JWT	Remove stock
/api/news	GET	-	Market news
/api/health	GET	-	Health check

Table 11.2 ML Service API Endpoints

Endpoint	Method	Purpose
/api/predict	POST	Generate LSTM predictions
/api/train	POST	Train/retrain model
/api/predict/history/{sym}	GET	Actual vs. predicted
/api/models	GET	List trained models
/api/health	GET	Service health check

Table 11.3 LSTM Model Architecture Summary

Layer	Configuration	Output Shape
Input	shape=(60, 1)	(None, 60, 1)
LSTM 1	128 units, return_seq=True	(None, 60, 128)
Dropout 1	rate=0.2	(None, 60, 128)
LSTM 2	64 units, return_seq=True	(None, 60, 64)
Dropout 2	rate=0.2	(None, 60, 64)
LSTM 3	32 units	(None, 32)
Dropout 3	rate=0.2	(None, 32)
Dense 1	16 units, ReLU	(None, 16)
Dense 2	1 unit, Linear	(None, 1)

REFERENCES

Bao W., Yue J., and Rao Y. (2017), *A deep learning framework for financial time series using stacked autoencoders and long-short term memory, PLOS ONE, vol. 12, no. 7.*

This paper proposes a novel deep learning framework combining stacked autoencoders for feature extraction with LSTM networks for stock prediction, demonstrating R-squared scores above 0.90 on major market indices.

Chong E., Han C., and Park F. C. (2017), *Deep learning networks for stock market analysis and prediction, Expert Systems with Applications, vol. 83, pp. 187-205.*

This study provides comprehensive benchmarking of deep learning architectures for financial prediction, validating the use of multi-layer networks for capturing complex market patterns.

Express.js Documentation (2024), *Express - Fast, unopinionated, minimalist web framework for Node.js. Available: https://expressjs.com/*

Express.js is the industry-standard Node.js web framework used in this project for REST API routing, middleware composition, and HTTP request handling.

FastAPI Documentation (2024), *FastAPI - Modern, fast web framework for building APIs with Python. Available: https://fastapi.tiangolo.com/*

FastAPI provides the high-performance async framework used to serve the LSTM prediction model as a RESTful API with automatic OpenAPI documentation.

Fischer T. and Krauss C. (2018), *Deep learning with long short-term memory networks for financial market predictions, European Journal of Operational Research, vol. 270, no. 2, pp. 654-669.*

This comprehensive benchmark study demonstrates that LSTM networks consistently outperform traditional ML models for stock prediction across S&P 500 constituents.

Hochreiter S. and Schmidhuber J. (1997), *Long Short-Term Memory, Neural Computation, vol. 9, no. 8, pp. 1735-1780.*
The foundational paper introducing the LSTM architecture with input, forget, and output gates that solve the vanishing gradient problem in recurrent neural networks.

Moghar A. and Hamiche M. (2020), *Stock Market Prediction Using LSTM Recurrent Neural Network, Procedia Computer Science, vol. 170, pp. 1168-1173.*
This study validates stacked LSTM architectures with decreasing unit counts and Dropout regularization for financial time-series forecasting.

MongoDB Documentation (2024), *MongoDB - The application data platform. Available: https://www.mongodb.com/docs/*
MongoDB is the document-oriented NoSQL database used in this project for flexible schema design and horizontal scalability.

React Documentation (2024), *React - A JavaScript library for building user interfaces, Meta Platforms, Inc. Available: https://react.dev/*
React 18 provides the component-based frontend framework with virtual DOM, hooks, and context API used throughout this project.

Selvin S. et al. (2017), *Stock price prediction using LSTM, RNN and CNN-sliding window model, ICACCI, pp. 1643-1647.*
This paper compares RNN, LSTM, and CNN approaches for stock prediction, establishing that LSTM networks achieve superior accuracy for time-series forecasting.

TensorFlow Documentation (2024), *TensorFlow - An end-to-end open source machine*

learning platform, Google. Available: https://www.tensorflow.org/

TensorFlow with Keras API provides the deep learning framework used to construct, train, and deploy the LSTM prediction model.

EXPERIENCE

The development of the AI-Powered Stock Market Analysis Platform provided invaluable hands-on experience across multiple domains of software engineering and artificial intelligence. Working on this project as an individual developer required managing the full software development lifecycle - from requirements analysis and system design through implementation, testing, and documentation.

The project environment was primarily self-hosted using a personal development workstation. The open-source community provided exceptional support through comprehensive documentation for React, TensorFlow, FastAPI, Express.js, and MongoDB. The availability of free-tier cloud services (MongoDB Atlas, GitHub) enabled collaborative development workflows and data persistence without incurring costs.

Key technical skills developed during this project include: deep learning model architecture design and hyperparameter tuning using TensorFlow/Keras; full-stack web development with React, Node.js, and Express; RESTful API design and microservices architecture; real-time data streaming using WebSocket protocols (Socket.IO); NoSQL database design with MongoDB; and secure authentication implementation using JWT and bcrypt.

The most challenging aspect of the project was optimizing the LSTM model to achieve consistent prediction accuracy across different stock symbols with varying price ranges and volatility profiles. This required extensive experimentation with sequence lengths, layer configurations, dropout rates, and learning rates over multiple development sprints.

The cooperation of the faculty members at the Department of Computer Science And Engineering, particularly Prof. Ruchi Patel, was instrumental in providing guidance on

research methodology, system design best practices, and thesis preparation. Their feedback during periodic reviews helped refine both the technical implementation and the academic documentation.